

Grobbit Language Reference

(Version 1.6)

In the compiler course you will write a compiler for the custom-made programming language **Grobbit**. It is a proper subset of the *Go* programming language (for the *Go* specifications, please look at the link <https://go.dev/ref/spec>).

The language has the following **literals**: *bool*, *integer*, *float*, and *string*. A *bool* literal is either *true* or *false*, which are reserved keywords (unlike in Go where they are predefined identifiers). *Integer* and *float* literals are the same as in Go except they are only decimal, they are typed, and thousand separators are not allowed. A *string* literal must be enclosed by double-quotes and consists of a sequence of Unicode-char and the following escaped-chars `\t`, `\n`, `\\`, and `\"`.

Identifier names and **keywords** are the same as in Go. Note even though some Go keywords are not used in Grobbit, then your lexical analyser must match all Go keywords (opposed to an identifier), otherwise Grobbit would not be a proper subset of Go!

As for allowed **operators** and **punctuation** marks, see the following grammar. In particular, opposed to Go, none of the bitwise operators are allowed, nor the increase (`++`), decrease (`--`), and arrow (`->`) operators, nor brackets (`[ ]`), colon (`:`), or period (`.`), nor the special tilde (`...`) token.

Please read the Go specification for more details on literals, identifiers, keywords, operators, and punctuation marks.

Grammar

<program> ::= **package** <pkg_name>; { <import_decl>; } { (<decl> | <func_decl>); }

<pkg_name> ::= <identifier>

<import_decl> ::= **import** (<import_spec> | { <import_spec>; })

<import_spec> ::= [<pkg_name>] **STRING**

<decl> ::= (<const_decl> | <var_decl>)

<const_decl> ::= **const** (<const_spec> | { <const_spec>; })

<const_spec> ::= <identifiers> <type> = <basic_literals>

<var_decl> ::= **var** (<var_spec> | { <var_spec>; })

<var_spec> ::= <identifiers> <type> [= <basic_literals>]

<basic_literals> ::= <basic_literal> { , <basic_literal> }

<basic_literal> ::= **INT** | **FLOAT** | **BOOL**

<func_decl> ::= **func** <identifier> ([<param_decl>]) [<type>] <block>

<param_decl> ::= <param_spec> { , <param_spec> }

<param_spec> ::= <identifiers> <type>

<identifiers> ::= <identifier> { , <identifier> }

<identifier> ::= **IDENTIFIER**

<type> ::= <identifier>

<block> ::= { { <statement>; } }

<statement> ::= <decl>
| <simple_stmt>
| <return_stmt> | <break_stmt> | <cont_stmt>
| <block>
| <if_stmt> | <for_stmt>

<simple_stmt> ::= <expr_stmt> | <assign_stmt> | <shv_decl_stmt>

<return_stmt> ::= **return** [<expr>]

<cont_stmt> ::= **continue**

<break_stmt> ::= **break**

<if_stmt> ::= **if** [<simple_stmt>;] <cond> <block> [**else** (<if_stmt> | <block>)]

<for_stmt> ::= **for** [<cond> | <for_clause>] <block>

<for_clause> ::= [<simple_stmt>;] [<cond>;] [<simple_stmt>]

<cond> ::= <expr>

<shv_decl_stmt> ::= <identifiers> := <expressions>

<assign_stmt> ::= <identifiers> = <expressions>

<expr_stmt> ::= <call_expr>

<expressions> ::= <expr> { , <expr> }

<expr> ::= <unary_expr> | <expr> <binary_op> <expr>

<unary_expr> ::= <primary_expr> | <unary_op> <unary_expr>

<binary_op> ::= || | **&&** | <rel_op> | <add_op> | <mul_op>

<unary_op> ::= + | - | !

<rel_op> ::= == | != | < | <= | > | >=

<add_op> ::= + | -

<mult_op> ::= * | / | %

<primary_expr> ::= <basic_literal>
| [<pkg_name> .] <identifier>
| <call_expr>
| (<expr>)

<call_expr> ::= [<pkg_name> .] <identifier> ([<expressions>])